



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

What is a programming language?



Learning Outcome:

Understands systems theories, business processes and basic concepts on IT.
Create a map of processes that allows identifying areas in an information system, using IT tools to generate reports based on the company's needs.

Ask your partner:

- Why do you think there are so many different programming languages?
- Do you think programming languages are any different from natural languages? Why?

Can Do Statements:

- I can understand written information about the history and development of programming languages.
- I can write about programming language elements.
- I can use vocabulary related to a programming language.



Let's Explore!

Professor Smith is explaining to his students what a programming language is.



Instructions: Read the text below.

Janine: Excuse me, professor. Can you tell us if HTML is a programming language? I know the L stands for language, but is it really doing something?

Professor: Well, that is a really good question, Janine. In order to answer that, I want to ask you something, what is a programming language?

Janine: Well, according to the text I'm reading, it says a programming language is a set of rules and algorithms that programs follow to carry out tasks.

Professor: Exactly, that means that all programming languages follow some processes and logic to express all kinds of algorithms. Therefore, we can say that programming languages have specific elements or traits, such as a function and a target as well as expressive power, syntax and semantics; for example, when you are installing a printer or a device, it is a program that is creating a postscript program to control such devices.

Janine: So, it means that HTML can't be a programming language, right?

Professor: That's correct, Janine. Programming languages are Turing complete, that is, languages that implement all kinds of algorithms; for example, C++, C#, Java, JavaScript, and so on; while HTML is just a markup language.

Janine: But, as far as I know, HTML also shares similar elements, like the syntax and semantics.

Professor: It's true, but when you are writing code within HTML, the syntax and the semantics of the markup language only allows you to display what you are writing, not to process the algorithms.

Janine: Sorry, professor. Can we take note of that information you are writing on the board?

Professor: Sure, Remember that even though programming languages and markup languages have syntax and semantic, they both are different types of systems; Turing complete languages follow algorithms and can create other programs, while markup languages can only show what you write within the html tags.





Let's Practice!

What are Professor Smith and his students doing to understand what programming languages are?

Instructions: Use the words from the word bank to complete each sentence. There is only one possible option. Read the sentences aloud and compare with an instructor.

Word Bank

is explaining / is reading / is taking / is playing / is recording

1. Janine _____ an article about syntax and semantics in programming languages.
2. Mr. Smith _____ the differences between Turing complete and markup languages.
3. Janine _____ notes to understand that programming languages, such as C++, have some traits like function and target.



Let's Practice!

Janine is reading more about programming languages.

Instructions: Organize the mystery sentences. Read the sentences aloud. Check your pronunciation with your instructor.

1. HTML, / can't / Markup / languages, / be / as / programming / languages. / such

2. can / as / a / algorithms. / programming / C++ / all / language / implement / it / is

3. programming / express / language / can / A / algorithms.



Extension Activity

Let's talk about programming languages.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for information about control flow statements in programming languages.
- Create a poster where you share some elements and important ideas.
- Use "can" and present continuous.
- Share this information with a classmate.

Self-Assessment

Yes

No

1. I read information about programming languages.

☐ ☐

2. I create a poster about important ideas related to programming languages.

☐ ☐

3. I practice the use of can and present continuous.

☐ ☐

4. I share my work with my classmates.

☐ ☐


Target Words

- **display** (verb): to show something on the screen of the computer
e.g. The syntax and the semantics of the markup language only allows you to display what you are writing, not to process the algorithms.
- **carry out** (phrasal verb): to perform an action
e.g. It says a programming language is a set of rules and algorithms that programs follow to carry out tasks.
- **algorithm** (noun): mathematical instructions or rules that help computers to carry out tasks
e.g. Exactly, that means that all programming languages follow some processes and logic to express all kinds of algorithms.
- **Turing complete** (adjective): any programming language that can implement a full set of algorithms
e.g. Programming languages are Turing complete, that is, languages that implement all kinds of algorithms; for example, C++, C#, Java, or JavaScript.
- **markup language** (noun): computing languages that define the structure of the language but do not carry out any algorithm
e.g. Turing complete languages follow algorithms and can create other programs, while markup languages can only show what you write within the html tags.



Learning Points

Present Continuous: Current Actions – All Forms

Use **the present continuous tense** to express ideas or actions that happen at the moment of speaking.

You can use the present continuous or progressive tense to express ongoing ideas (ideas that are in progress). To do so, you can use the following structures; depending on the form, you are using it (affirmative, interrogative, negative).

AFFIRMATIVE:

In order to make a present continuous sentence, you need to take into account that in this time tense, the verb to be works as the auxiliary verb and the main verb (action verb) goes with the prefix -ing.

Subject + **to be** + **Verb** + **ing** + Complement

Examples:

***I'm reading** about programming languages and their history.*

*Some software companies **are creating** new languages to make their software privative.*

NEGATIVE

Remember that in negative statements what you deny is the auxiliary verb (to be).

Subject + **To be** + **not** + Verb + **ing** + complement.

Examples:

*Most companies **are not working** with new programming languages but standard ones, like C++ or Java.*

*Our company **is not implementing** Java but JavaScript.*

INTERROGATIVE

Remember that you can use open-ended (wh) or close-ended (yes/no) questions; either way, the structure you need to use is the same.

(**Wh**) + **To be** + Subject + Verb + **ing** + complement + ?





Learning Points

Examples:

What programming language **are** you **implementing** to create the company's app?

Are you still **working** on that app?

Remember that for close-ended questions, you can use **short answers**:

Yes, + Subject + **is/are**

No, + Subject + **is not/are not**

Examples:

Are you still **working** on that app?

Yes, I am.

Is she **programming** with Java instead of JavaScript?

No, she isn't.

When you are installing a printer or a device, it is really a program that is creating a postscript program to control such devices.

Examples

- *I know the L stands for language, but is it really doing something?*
- *Well, according to the text I'm reading, it says a programming language is a set of rules and algorithms that programs follow to carry out tasks.*
- *Can we take note of that information you are writing on the board?*



Learning Points

Programming Language: Elements

A programming language is all the commands that a person use to communicate with programs; therefore, there are some elements that you need to understand, such as the syntax or semantics of a program.

You need to understand that, by now, there are hundreds of programming languages that companies implement to create their software, and that many other companies and programmers are creating many more, as some companies want to protect their information.

Regardless of the language you are implementing to create your software, some of the most basic elements you need to take into account are: syntax and semantics.

SYNTAX

Syntax is the same form the programming language uses. Some languages are textual and use words, numbers, and special characters. Others can be graphical as they use visual relationships between symbols to represent the program.

SEMANTICS

Semantics are the same meaning the language conveys or represents by means of its syntax, that is, the connotation the sum of symbols (textual or graphical) have.

If you compare programming languages to natural languages, you can realize that they are not too different in those two aspects.

For example, in a natural language (English), you need to use the alphabet to represent words (syntax). However, you need to combine those words in a specific order, following some grammar rules, to convey meaning (semantics).

Example:

Software languages programming create.

In the previous sentence, we are making a correct use of syntax; however, there is no meaning beyond those symbols.

In order to create meaning, you need to rearrange the words like this:

Programming languages create software.

Similarly, programming languages have symbols that represent something (syntax) and organization to convey meaning (semantics).

Example:

```
<div id="mw-page-base" class="noprint"></div>
```

